

15.1

Monolith vs Microservices

vs Modular Monolith

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

What is the Problem?

Imagine you are building a shopping website. You could build everything in one big program (a monolith), or split it into many small programs (microservices). Each approach has trade-offs. This chapter explains all three styles so you can choose wisely.

■ **Simple Analogy:** A monolith is like a Swiss Army knife — one tool, does everything. Microservices are like a full kitchen — each tool specialized. A modular monolith is like a well-organized toolbox.

The Three Styles

Monolith	One big application. All code is deployed together. One database. Simple to start but hard to scale.
Microservices	Many small applications, each doing one job. Each has its own database. Independent deploys.
Modular Monolith	One deployable app BUT with strict internal boundaries. Best of both worlds for small teams.

Diagram — Three Styles Side by Side

MONOLITH	MODULAR MONOLITH	MICROSERVICES
[Users] [Orders] [Payments] [Inventory] – all in one jar –	[Users Module] [Orders Module] [Payments Module] [Inventory Module] – strict walls, one jar –	[User Svc:DB1] [Order Svc:DB2] [Payment Svc:DB3] – separate jars/containers –
One DB One Deploy One team	Shared DB One Deploy Module teams	DB per svc Indep. Deploys Indep. teams

Figure 1: Three architectural styles compared

When to Use Each

Situation	Best Choice	Why
Small team, new project	Monolith	Fast to build
Growing team (10-50 devs)	Modular Monolith	Clean, no network overhead

Large org (50+ devs / teams)	Microservices	Team autonomy, scale independently
Start-up MVP	Monolith	Simplicity first

■ **Real-World Example:** Amazon started as one monolith (1994). By 2002 Jeff Bezos mandated all teams expose data through services — this became AWS. Today they run 700+ microservices. The migration took 5+ years using the Strangler Fig pattern.

Key Takeaway

There is no universally 'best' style. Start simple (monolith). Add boundaries (modular). Split only when team size and scaling genuinely demand it. Premature microservices add enormous operational cost with little benefit.

Interview Tip: When asked 'monolith vs microservices', always mention team size, deployment frequency, and operational maturity as the deciding factors.